

Rempart Physique Linux (Physique, Intégrité et MFA)

- Comprendre et bloquer les attaques par accès physique ou console (Hyperviseur).
- Sécuriser le chargeur d'amorçage (GRUB) par cryptographie.
- Mettre en œuvre un contrôle d'intégrité des fichiers (FIM) pour détecter les Rootkits.
- Déployer l'Authentification Multi-Facteurs (MFA/TOTP) sur le service SSH.

1. Sécurité Physique et du Chargeur d'Amorçage (GRUB)

Le durcissement du système d'exploitation perd tout son sens si un attaquant dispose d'un accès physique à la machine, ou d'un accès à la console de la machine virtuelle (via vCenter, Proxmox, etc.).

1.1 La vulnérabilité du Boot

Au démarrage du serveur, le chargeur d'amorçage **GRUB** s'affiche pendant quelques secondes. Par défaut, n'importe quel utilisateur ayant accès au clavier peut appuyer sur la touche **e** (Éditer) et modifier les paramètres passés au noyau Linux.

- **L'attaque classique** : Ajouter `init=/bin/bash` ou le mot `single` à la fin de la ligne de boot du noyau.
- **Le résultat** : Le système démarre instantanément sur un shell en tant que `root`, sans jamais demander de mot de passe. L'attaquant peut alors changer le mot de passe root avec `passwd` et redémarrer la machine.

1.2 La parade : Verrouiller GRUB (PBKDF2)

Pour empêcher cette manipulation, l'ANSSI recommande de verrouiller le menu GRUB avec un mot de passe haché de manière robuste (PBKDF2).

1. On génère un mot de passe haché via la commande `grub-mkpasswd-pbkdf2`.
2. On déclare un super-utilisateur GRUB et on y associe le hash dans le fichier `/etc/grub.d/40_custom`.
3. On met à jour la configuration avec `update-grub`. *Résultat* : Le serveur démarrera normalement de manière automatique, mais toute tentative d'édition ou de démarrage en mode de secours exigera le mot de passe.

1.3 Bloquer les supports de stockage USB

Un attaquant ayant un accès physique peut brancher une clé USB malveillante (ex: Rubber Ducky ou exfiltration de données). On peut interdire matériellement le montage des clés USB en plaçant le module noyau `usb-storage` sur liste noire (Blacklist). Dans `/etc/modprobe.d/blacklist.conf` :

- `install usb-storage /bin/true`

2. Contrôle d'Intégrité des Fichiers (FIM)

Comment détecter un attaquant très discret (ou un *Rootkit*) qui aurait remplacé la commande `/bin/ls` par une version malveillante cachant ses propres fichiers ? Un antivirus classique basé sur des signatures ne verrait rien si le code est inédit.

2.1 Le concept du FIM (File Integrity Monitoring)

La solution consiste à prendre une "photographie" cryptographique (empreinte) de tous les fichiers vitaux du système à un instant T (lorsque le système est sain). Ensuite, on compare régulièrement l'état du système à cette photographie. Si un seul octet d'un fichier sensible a été modifié, son empreinte change, et une alerte est déclenchée.

2.2 L'outil AIDE (Advanced Intrusion Detection Environment)

AIDE est l'outil standard pour cette tâche sous Linux (similaire à *Tripwire*).

- **Étape 1 (Initialisation)** : AIDE scanne les dossiers critiques (définis dans `/etc/aide/aide.conf`) et calcule les hashés (SHA512) de chaque binaire. Il génère une base de données de référence (`aide.db.new`).
- **Étape 2 (Mise en sécurité)** : Cette base de référence doit idéalement être copiée sur un serveur distant en lecture seule.
- **Étape 3 (L'Audit régulier)** : Tous les jours, la commande `aide --check` est lancée. Elle recalcule les empreintes et signale la moindre modification non autorisée.

3. L'Authentification Multi-Facteurs (MFA via PAM)

Le vol de mots de passe (Phishing, fuite de données, Keyloggers) est la première cause de compromission. Le durcissement ultime de l'accès SSH consiste à exiger une preuve de possession (un téléphone) en plus de la preuve de connaissance (le mot de passe).

3.1 Implémentation via Google Authenticator

Linux gère ce mécanisme grâce à l'architecture PAM (*Pluggable Authentication Modules*). Nous utilisons le standard **TOTP** (Time-based One-Time Password) qui génère un code à 6 chiffres changeant toutes les 30 secondes.

1. **Le paquet** : `libpam-google-authenticator`.
2. **L'enrôlement** : L'utilisateur lance la commande `google-authenticator`. Un QR Code s'affiche dans le terminal, qu'il scanne avec son application smartphone (Google Authenticator, Microsoft Authenticator, FreeOTP, etc.).
3. **L'activation PAM** : On ajoute la directive `auth required pam_google_authenticator.so` dans le fichier `/etc/pam.d/sshd`.
4. **L'activation SSH** : On modifie `/etc/ssh/sshd_config` pour autoriser l'interactivité au clavier (`KbdInteractiveAuthentication yes`).

Désormais, lors de la connexion, le Bastion demandera le mot de passe habituel, suivi d'un `Verification code`: que seul le possesseur du téléphone pourra fournir. C'est un prérequis fort pour les environnements de type "Tier 0" et les Bastions d'administration exposés.

Fin de la séquence Linux. Le Bastion est désormais sécurisé contre les menaces logiques, réseau et physiques.